

ROADRESQ

ER Diagram + Data Dictionary

Version 1.0 • Production-Oriented Database Reference

Database: PostgreSQL + PostGIS

Purpose: Transactional, geospatial and auditable automotive service data model

USER → VEHICLE → BOOKING → PROVIDER → JOB → ESTIMATE → INVOICE → PAYMENT →
REVIEW

Document Control

Field	Value
Document	ER Diagram + Data Dictionary
Product	RoadResQ
Version	1.0
Status	Database design baseline
Primary database	PostgreSQL + PostGIS
Audience	Backend, database, frontend, QA, DevOps and data/analytics engineers

1. Database Architecture Overview

RoadResQ uses a relational transactional model with PostgreSQL as the system of record. PostGIS extends PostgreSQL for location-aware provider discovery and routing-related data. Redis may support transient realtime coordination, but authoritative business records remain in PostgreSQL.

Core business relationship:

USER → VEHICLE → BOOKING → PROVIDER → JOB CARD → ESTIMATE → INVOICE → PAYMENT → REVIEW

2. ER Diagram — Logical Model

The following Mermaid ER definition is the canonical logical relationship reference for the database. It can be rendered in Mermaid-compatible documentation/tools.

```
erDiagram
    USERS ||--o{ VEHICLES : owns
    USERS ||--o{ PROVIDERS : operates
    PROVIDERS ||--o{ PROVIDER_DOCUMENTS : submits
    PROVIDERS ||--o{ PROVIDER_SERVICES : offers
    PROVIDERS ||--o{ PROVIDER_AVAILABILITY : has
    USERS ||--o{ BOOKINGS : creates
    VEHICLES ||--o{ BOOKINGS : used_for
    PROVIDERS ||--o{ BOOKINGS : assigned_to
    BOOKINGS ||--o{ BOOKING_LOCATIONS : has
    BOOKINGS ||--o{ BOOKING_STATUS_HISTORY : records
    PROVIDERS ||--o{ PROVIDER_LOCATION_UPDATES : sends
    BOOKINGS ||--o{ JOB_CARDS : creates
    JOB_CARDS ||--o{ INSPECTIONS : contains
    INSPECTIONS ||--o{ INSPECTION_ITEMS : contains
    BOOKINGS ||--o{ ESTIMATES : has
    ESTIMATES ||--o{ ESTIMATE_ITEMS : contains
    BOOKINGS ||--o{ INVOICES : generates
    INVOICES ||--o{ INVOICE_ITEMS : contains
    INVOICES ||--o{ PAYMENTS : paid_by
    PROVIDERS ||--o{ PAYOUTS : receives
    BOOKINGS ||--o{ REVIEWS : receives
    USERS ||--o{ NOTIFICATIONS : receives
    BOOKINGS ||--o{ MESSAGES : contains
    USERS ||--o{ SUPPORT_TICKETS : creates
    BOOKINGS ||--o{ SUPPORT_TICKETS : concerns
    USERS ||--o{ AUDIT_LOGS : performs
```

3. Relationship & Cardinality Rules

Relationship	Cardinality	Rule
User → Vehicles	1 : N	A customer may own multiple vehicles.
User → Provider	1 : 0..1	A user may operate one provider profile in the primary model.
Provider → Documents	1 : N	Provider can submit multiple verification documents/revisions.
Provider → Services	1 : N	Provider can offer multiple service capabilities.
User → Bookings	1 : N	Customer may create multiple bookings.
Vehicle → Bookings	1 : N	A vehicle may appear in many service bookings.
Provider → Bookings	1 : N	Provider may handle many bookings over time.
Booking → Status History	1 : N	Every important state transition is recorded.
Booking → Job Card	1 : 0..1	Operational job record created for eligible bookings.
Job → Inspections	1 : N	Inspection records can contain multiple inspection sessions/items.
Booking → Estimates	1 : N	Revisions are represented as separate estimates or versions.
Booking → Invoice	1 : 0..1	A finalized booking normally produces one final invoice.
Invoice → Payments	1 : N	Supports retries/partial or multiple payment records if business rules permit.
Booking → Review	1 : 0..1	One eligible customer review per booking in MVP.

4. Naming & Schema Conventions

- Table and column names use snake_case.
- Primary keys should use UUID/ULID-style identifiers suitable for distributed systems.
- Foreign keys use the referenced entity name plus _id.
- created_at and updated_at are recommended on mutable business tables.
- Use timestamptz for event/business timestamps.
- Use numeric/decimal for money; never floating point.
- Use enums or controlled lookup/reference tables for stable state values.
- Use geography(Point, 4326) or geometry with an explicitly defined SRID for location fields.
- Soft deletion/deactivation should be used where historical records must remain intact.
- Financial and audit records should be append-only or versioned after finalization.

5. Data Dictionary — Identity & Customer

5.1 users

Column	Type	Key	Null	Description
id	uuid	PK	NO	Unique user identifier
role	varchar/enum	—	NO	CUSTOMER / PROVIDER / SUPPORT / ADMIN
phone	varchar	UNIQUE	YES	Verified phone number where used
email	varchar	UNIQUE	YES	Email address where used
full_name	varchar	—	NO	Display/legal name as configured
status	varchar/enum	—	NO	Account lifecycle state
created_at	timestampz	—	NO	Creation timestamp
updated_at	timestampz	—	NO	Last update timestamp
last_login_at	timestampz	—	YES	Most recent successful login

5.2 vehicles

Column	Type	Key	Null	Description
id	uuid	PK	NO	Vehicle identifier
user_id	uuid	FK → users.id	NO	Owner/customer
registration_number	varchar	INDEX/UNIQUE	NO	Vehicle registration/identifier
make	varchar	—	NO	Manufacturer
model	varchar	—	NO	Model
variant	varchar	—	YES	Variant/trim
fuel_type	varchar/enum	—	NO	PETROL/DIESEL/CNG/EV/HYBRID/etc.
year	smallint	—	YES	Manufacturing/model year
status	varchar/enum	—	NO	ACTIVE/INACTIVE
created_at	timestampz	—	NO	Creation timestamp
updated_at	timestampz	—	NO	Last update timestamp

6. Data Dictionary — Provider Domain

6.1 providers

Column	Type	Key	Null	Description
id	uuid	PK	NO	Provider identifier
user_id	uuid	FK → users.id	NO	Provider account owner
business_name	varchar	—	NO	Garage/mechanic/service business name
provider_type	varchar/enum	—	NO	MECHANIC/GARAGE/TOWING/etc.
phone	varchar	—	YES	Operational contact
verification_status	varchar/enum	INDEX	NO	PENDING/VERIFIED/REJECTED/SUSPENDED
service_radius_km	numeric	—	YES	Configured service radius
rating_avg	numeric	—	NO	Derived/cached rating
status	varchar/enum	INDEX	NO	ACTIVE/OFFLINE/SUSPENDED
created_at	timestamptz	—	NO	Creation timestamp
updated_at	timestamptz	—	NO	Last update timestamp

6.2 provider_documents

Column	Type	Key	Null	Description
id	uuid	PK	NO	Document record
provider_id	uuid	FK	NO	Provider
document_type	varchar/enum	—	NO	Type of verification document
storage_key	varchar	—	NO	Private object-storage reference
verification_status	varchar/enum	—	NO	PENDING/APPROVED/REJECTED
reviewed_by	uuid	FK → users.id	YES	Admin reviewer
reviewed_at	timestamptz	—	YES	Review timestamp
created_at	timestamptz	—	NO	Upload timestamp

6.3 provider_services

Column	Type	Key	Null	Description
id	uuid	PK	NO	Capability record
provider_id	uuid	FK	NO	Provider
service_type	varchar/enum	INDEX	NO	Supported service category
is_active	boolean	—	NO	Whether currently offered
base_price	numeric	—	YES	Optional configured base price
created_at	timestamptz	—	NO	Creation timestamp

7. Data Dictionary — Availability & Booking

7.1 provider_availability

Column	Type	Key	Null	Description
id	uuid	PK	NO	Availability record
provider_id	uuid	FK	NO	Provider
day_of_week	smallint	—	NO	0–6 or configured convention
start_time	time	—	NO	Operating start
end_time	time	—	NO	Operating end
is_active	boolean	—	NO	Schedule enabled

7.2 bookings

Column	Type	Key	Null	Description
id	uuid	PK	NO	Booking identifier
customer_id	uuid	FK → users.id	NO	Requesting customer
vehicle_id	uuid	FK → vehicles.id	NO	Vehicle requiring service
provider_id	uuid	FK → providers.id	YES	Assigned provider
service_type	varchar/enum	INDEX	NO	Requested service
status	varchar/enum	INDEX	NO	Authoritative lifecycle state
priority	varchar/enum	—	NO	Normal/urgent or configurable
scheduled_at	timestamptz	INDEX	YES	Scheduled service time
description	text	—	YES	Customer problem description
created_at	timestamptz	INDEX	NO	Creation time
updated_at	timestamptz	—	NO	Last update

7.3 booking_locations

Column	Type	Key	Null	Description
id	uuid	PK	NO	Location record
booking_id	uuid	FK	NO	Booking
location_type	varchar/enum	—	NO	REQUEST/DESTINATION/OTHER
point	geography(Point,4326)	GIST	NO	Latitude/longitude
address_text	text	—	YES	Human-readable address
accuracy_m	numeric	—	YES	Reported GPS accuracy
created_at	timestamptz	—	NO	Captured timestamp

7.4 booking_status_history

Column	Type	Key	Null	Description	
id	uuid	PK	NO	History event	
booking_id	uuid	FK	INDEX	NO	Booking
previous_status	varchar/enum	—	YES	Previous state	
new_status	varchar/enum	—	NO	New state	
actor_user_id	uuid	FK	YES	User/system actor	
reason	text	—	YES	Transition reason	
created_at	timestamptz	INDEX	NO	Transition timestamp	

8. Data Dictionary — Realtime & Job Domain

8.1 provider_location_updates

Column	Type	Key	Null	Description	
id	uuid	PK	NO	Location update ID	
provider_id	uuid	FK	INDEX	NO	Provider
booking_id	uuid	FK	INDEX	YES	Active booking context
position	geography(Point,4326)	GIST	NO	Provider position	
accuracy_m	numeric	—	YES	Location accuracy	
direction	numeric	—	YES	Direction	
speed_mps	numeric	—	YES	Speed	

Column	Type	Key	Null	Description
created_at	timestampz	INDEX	NO	Device timestamp

8.2 job_cards

Column	Type	Key	Null	Description
id	uuid	PK	NO	Job card
booking_id	uuid	FK/UNIQUE	NO	Associated booking
provider_id	uuid	FK	NO	Provider
arrival_otp_verified	boolean	—	NO	Arrival verification result
started_at	timestampz	—	YES	Work start
completed_at	timestampz	—	YES	Work completion
notes	text	—	YES	Operational notes
created_at	timestampz	—	NO	Creation timestamp
updated_at	timestampz	—	NO	Last update

8.3 inspections / inspection_items

Entity	Important fields	Purpose
inspections	id, job_card_id, inspector_id, notes, created_at	Inspection session/header
inspection_items	id, inspection_id, component, condition, severity, notes, media_key	Individual inspected component/finding

9. Data Dictionary — Estimates & Billing

9.1 estimates

Column	Type	Key	Null	Description
id	uuid	PK	NO	Estimate ID/version
booking_id	uuid	FK/INDEX	NO	Booking
provider_id	uuid	FK	NO	Provider creating estimate
version	integer	—	NO	Estimate revision number
status	varchar/enum	—	NO	DRAFT/PENDING/APPROVED/REJECTED/SUPERSEDED
subtotal	numeric	—	NO	Pre-tax amount
tax_amount	numeric	—	NO	Tax total
total_amount	numeric	—	NO	Customer total
approved_by	uuid	FK → users.id	YES	Customer approver
approved_at	timestampz	—	YES	Approval timestamp

9.2 estimate_items

Column	Type	Key	Null	Description
id	uuid	PK	NO	Estimate line
estimate_id	uuid	FK	NO	Estimate
description	text	—	NO	Part/labour/service description
quantity	numeric	—	NO	Quantity
unit_price	numeric	—	NO	Price per unit
tax_rate	numeric	—	YES	Applicable tax rate
line_total	numeric	—	NO	Calculated line amount

9.3 invoices

Column	Type	Key	Null	Description
id	uuid	PK	NO	Invoice identifier
booking_id	uuid	FK/UNIQUE	NO	Booking
invoice_number	varchar	UNIQUE	NO	Human-readable invoice number
subtotal	numeric	—	NO	Pre-tax total
tax_amount	numeric	—	NO	Tax total
discount_amount	numeric	—	NO	Discounts
total_amount	numeric	—	NO	Final amount
status	varchar/enum	INDEX	NO	DRAFT/ISSUED/VOID
issued_at	timestampz	—	YES	Issue timestamp

9.4 invoice_items / payments / payouts

Entity	Key fields	Purpose
invoice_items	invoice_id, description, quantity, unit_price, tax_rate, line_total	Invoice lines
payments	invoice_id, booking_id, amount, method, status, gateway_reference, idempotency_key	Customer payment records
payouts	provider_id, booking_id, amount, status, settlement_reference, paid_at	Provider settlement records

10. Data Dictionary — Engagement & Operations

Entity	Key fields	Purpose
reviews	id, booking_id, customer_id, provider_id, rating, comment, created_at	Customer feedback
notifications	id, user_id, type, title, body, channel, status, read_at, created_at	Notification lifecycle
messages	id, booking_id, sender_id, message_type, body, media_key, created_at	Booking communication
support_tickets	id, user_id, booking_id, category, priority, status, assigned_to, created_at	Customer/provider support
audit_logs	id, actor_user_id, action, entity_type, entity_id, metadata, ip_address, created_at	Security/admin audit trail

11. Keys, Constraints & Indexing Strategy

11.1 Primary/Foreign Keys

- Every transactional entity has a stable primary identifier.
- Foreign keys should enforce valid relationships unless an explicit archival strategy requires otherwise.
- Use UNIQUE constraints for identifiers such as user phone/email where applicable, vehicle registration where business rules require, invoice numbers and provider assignment constraints.

11.2 Recommended Indexes

Table	Index	Reason
vehicles	(user_id), registration_number	Ownership lookups and registration search
providers	(status, verification_status), geospatial location	Eligible-provider discovery
provider_services	(service_type, provider_id)	Capability filtering

Table	Index	Reason
bookings	(customer_id, created_at), (provider_id, status), (status, created_at)	Customer/provider/operations queries
booking_locations	GIST(point)	Geospatial lookup
provider_location_updates	(provider_id, recorded_at), GIST(point) where useful	Recent tracking/history
booking_status_history	(booking_id, created_at)	Timeline retrieval
payments	(invoice_id), (status, created_at), gateway_reference	Payment reconciliation
notifications	(user_id, read_at, created_at)	Unread/list queries
audit_logs	(entity_type, entity_id, created_at), (actor_user_id, created_at)	Audit investigations

12. Transaction & Integrity Rules

- Booking creation, assignment and critical state changes must use database transactions.
- Provider acceptance must be protected against concurrent requests.
- Estimate approval and invoice finalization must validate current booking state.
- Payment records must be idempotent and reconciled against gateway status.
- Financial totals should be calculated server-side and stored using exact numeric types.
- Database constraints complement application validation; neither should be relied on alone.
- Audit/history records should survive normal business-record deactivation.

13. Data Lifecycle & Retention

Data class	Recommended handling
Account data	Retain while account/business relationship requires; support controlled deletion/anonymization.
Vehicle/service history	Retain according to product/legal/operational policy; preserve necessary historical integrity.
Location data	Minimize precision/frequency and retention; active tracking should not become indefinite surveillance.
Provider documents	Private storage; retain only as required for verification/compliance.
Payments/invoices	Retain according to accounting/legal requirements; restrict access.
Messages/support	Retain according to support, safety and privacy policy.
Audit logs	Protect against modification; retention should support security investigations and policy.

14. Migration & Versioning Strategy

- All schema changes should be version-controlled migrations.
- Production migrations must be reviewed and tested against realistic data volumes.
- Destructive changes should follow expand → migrate → contract patterns where zero/low downtime is required.
- Application releases must remain compatible with the database during rolling deployments.
- Seed/reference data changes should be deterministic and auditable.

15. Database Security

- Database should not be publicly exposed; access through private network/security controls.
- Use separate credentials/roles with least privilege.
- Application roles should not receive unnecessary administrative database privileges.

- Encrypt backups and restrict backup access.
- Protect provider documents and other sensitive object-storage references.
- Do not store raw passwords, OTPs or unnecessary payment secrets.
- Audit privileged database access where platform infrastructure supports it.

16. Data Dictionary — Core Enum Reference

Enum / Controlled value	Example values
user_role	CUSTOMER, PROVIDER, SUPPORT, ADMIN
account_status	PENDING, ACTIVE, SUSPENDED, BLOCKED
provider_verification_status	PENDING, VERIFIED, REJECTED, SUSPENDED
provider_status	OFFLINE, ACTIVE, SUSPENDED
booking_status	REQUESTED, SEARCHING, PROVIDER_ASSIGNED, ACCEPTED, ON_THE_WAY, ARRIVED, INSPECTION, ESTIMATE_PENDING, APPROVED, IN_PROGRESS, COMPLETED, INVOICE_GENERATED, PAYMENT_COMPLETED, CLOSED, CANCELLED, EXPIRED, DISPUTED
estimate_status	DRAFT, PENDING, APPROVED, REJECTED, SUPERSEDED
payment_status	PENDING, PROCESSING, SUCCESS, FAILED, REFUNDED, RECONCILIATION_PENDING
payment_method	UPI, CASH, CARD, OTHER
ticket_status	OPEN, IN_PROGRESS, WAITING, RESOLVED, CLOSED

17. ER-to-API Traceability

Database domain	Primary API domain
users	/auth, /customers, /admin/users
vehicles	/customers/me/vehicles, /vehicles
providers	/providers, /admin/providers
provider_documents	/providers/me/documents, /admin/providers/{id}/verification
provider_services / availability	/providers/me/services, /providers/me/availability
bookings	/bookings, /provider/requests, /admin/bookings
booking_status_history	/bookings/{id}/status-history
provider_location_updates	/provider/location, /bookings/{id}/location
job_cards / inspections	/bookings/{id}/inspection, provider job APIs
estimates	/bookings/{id}/estimates, /estimates/{id}/approve
invoices / payments / payouts	/invoices, /payments, /providers/me/payouts
reviews	/bookings/{id}/review or reviews API
notifications / messages	/notifications, booking messaging APIs
support / audit	/support, /admin/tickets, /admin/audit-logs

18. Production Database Readiness Checklist

- All tables have clear ownership and primary keys.
- Foreign keys and uniqueness constraints reflect business rules.
- Geospatial columns have correct SRID and appropriate GIST indexes.
- Booking state transitions are protected by transactional logic.
- Financial amounts use exact numeric types.

- Audit/history records are preserved and queryable.
- Sensitive data is minimized and access controlled.
- Migrations are automated, tested and version controlled.
- Backup/restore has been tested.
- Indexes are reviewed using realistic query plans and data volumes.
- Retention and deletion policies are documented.
- Database monitoring and alerting are configured.

19. Final Database Model Summary

The RoadResQ data model is centered on the customer, vehicle and provider relationship and uses bookings as the central transaction. Operational entities extend the booking into dispatch history, live location, job cards, inspections and estimates. Financial entities then produce invoices, payments and provider payouts, while reviews, notifications, messages, support tickets and audit logs provide the surrounding operational layer. PostgreSQL/PostGIS remains authoritative for transactional and geospatial data, with Redis reserved for transient/realtime coordination.